

CHAPTER 9

INFERENCE IN FIRST-ORDER LOGIC

In which we define effective procedures for answering questions posed in first-order logic.

In this chapter, we describe algorithms that can answer any answerable first-order logic question. Section 9.1 introduces inference rules for quantifiers and shows how to reduce first-order inference to propositional inference, albeit at potentially great expense. Section 9.2 describes how **unification** can be used to construct inference rules that work directly with first-order sentences. We then discuss three major families of first-order inference algorithms: **forward chaining** (Section 9.3), **backward chaining** (Section 9.4), and **resolution-based theorem proving** (Section 9.5).

9.1 Propositional vs. First-Order Inference

One way to do first-order inference is to convert the first-order knowledge base to propositional logic and use propositional inference, which we already know how to do. A first step is eliminating universal quantifiers. For example, suppose our knowledge base contains the standard folkloric axiom that all greedy kings are evil:

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x).$$

From that we can infer any of the following sentences:

$$\begin{aligned} &\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John}) \\ &\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard}) \\ &\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John})). \\ &\vdots \end{aligned}$$

In general, the rule of **Universal Instantiation** (UI for short) says that we can infer any sentence obtained by substituting a **ground term** (a term without variables) for a universally quantified variable.¹

To write out the inference rule formally, we use the notion of **substitutions** introduced in Section 8.3. Let $\text{SUBST}(\theta, \alpha)$ denote the result of applying the substitution θ to the sentence α . Then the rule is written

$$\frac{\forall v \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

for any variable v and ground term g . For example, the three sentences given earlier are obtained with the substitutions $\{x/\text{John}\}$, $\{x/\text{Richard}\}$, and $\{x/\text{Father}(\text{John})\}$.

¹ Do not confuse these substitutions with the extended interpretations used to define the semantics of quantifiers in Section 8.2.6. The substitution replaces a variable with a term (a piece of syntax) to produce a new sentence, whereas an interpretation maps a variable to an object in the domain.

Existential
Instantiation

Similarly, the rule of **Existential Instantiation** replaces an existentially quantified variable with a single *new constant symbol*. The formal statement is as follows: for any sentence α , variable v , and constant symbol k that does not appear elsewhere in the knowledge base,

$$\frac{\exists v \alpha}{\text{SUBST}(\{v/k\}, \alpha)}.$$

For example, from the sentence

$$\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

we can infer the sentence

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

as long as C_1 does not appear elsewhere in the knowledge base. Basically, the existential sentence says there is some object satisfying a condition, and applying the existential instantiation rule just gives a name to that object. Of course, that name must not already belong to another object. Mathematics provides a nice example: suppose we discover that there is a number that is a little bigger than 2.71828 and that satisfies the equation $d(x^y)/dy = x^y$ for x . We can give this number the name e , but it would be a mistake to give it the name of an existing object, such as π . In logic, the new name is called a **Skolem constant**.

Skolem constant

Whereas Universal Instantiation can be applied many times to the same axiom to produce many different consequences, Existential Instantiation need only be applied once, and then the existentially quantified sentence can be discarded. For example, we no longer need $\exists x \text{Kill}(x, \text{Victim})$ once we have added the sentence $\text{Kill}(\text{Murderer}, \text{Victim})$.

9.1.1 Reduction to propositional inference

We now show how to convert any first-order knowledge base into a propositional knowledge base. The first idea is that, just as an existentially quantified sentence can be replaced by one instantiation, a universally quantified sentence can be replaced by the set of *all possible* instantiations. For example, suppose our knowledge base contains just the sentences

$$\begin{aligned} &\forall x \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x) \\ &\text{King}(\text{John}) \\ &\text{Greedy}(\text{John}) \\ &\text{Brother}(\text{Richard}, \text{John}). \end{aligned} \tag{9.1}$$

and that the only objects are *John* and *Richard*. We apply UI to the first sentence using all possible substitutions, $\{x/\text{John}\}$ and $\{x/\text{Richard}\}$. We obtain

$$\begin{aligned} &\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John}) \\ &\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard}). \end{aligned}$$

Next replace ground atomic sentences, such as $\text{King}(\text{John})$, with proposition symbols, such as JohnIsKing . Finally, apply any of the complete propositional algorithms in Chapter 7 to obtain conclusions such as JohnIsEvil , which is equivalent to $\text{Evil}(\text{John})$.

This technique of **propositionalization** can be made completely general, as we show in Section 9.5. However, there is a problem: when the knowledge base includes a function symbol, the set of possible ground-term substitutions is infinite! For example, if the knowledge base mentions the *Father* symbol, then infinitely many nested terms such as $\text{Father}(\text{Father}(\text{Father}(\text{John})))$ can be constructed.

Propositionalization

Fortunately, there is a famous theorem due to Jacques Herbrand (1930) to the effect that if a sentence is entailed by the original, first-order knowledge base, then there is a proof involving just a *finite* subset of the propositionalized knowledge base. Since any such subset has a maximum depth of nesting among its ground terms, we can find the subset by first generating all the instantiations with constant symbols (*Richard* and *John*), then all terms of depth 1 (*Father(Richard)* and *Father(John)*), then all terms of depth 2, and so on, until we are able to construct a propositional proof of the entailed sentence.

We have sketched an approach to first-order inference via propositionalization that is **complete**—that is, any entailed sentence can be proved. This is a major achievement, given that the space of possible models is infinite. On the other hand, we do not know until the proof is done that the sentence *is* entailed! What happens when the sentence is *not* entailed? Can we tell? Well, for first-order logic, it turns out that we cannot. Our proof procedure can go on and on, generating more and more deeply nested terms, but we will not know whether it is stuck in a hopeless loop or whether the proof is just about to pop out. This is very much like the halting problem for Turing machines. Alan Turing (1936) and Alonzo Church (1936) both proved, in rather different ways, the inevitability of this state of affairs. *The question of entailment for first-order logic is semidecidable*—that is, algorithms exist that say yes to every entailed sentence, but no algorithm exists that also says no to every nonentailed sentence.

9.2 Unification and First-Order Inference

The sharp-eyed reader will have noticed that the propositionalization approach generates many unnecessary instantiations of universally quantified sentences. We'd rather have an approach that uses just the one rule, reasoning that $\{x/John\}$ solves the query *Evil*(*x*) as follows: given the rule that greedy kings are evil, find some *x* such that *x* is a king and *x* is greedy, and then infer that this *x* is evil. More generally, if there is some substitution θ that makes each of the conjuncts of the premise of the implication identical to sentences already in the knowledge base, then we can assert the conclusion of the implication, after applying θ . In this case, the substitution $\theta = \{x/John\}$ achieves that aim. Now suppose that instead of knowing *Greedy(John)*, we know that *everyone* is greedy:

$$\forall y \text{ Greedy}(y). \quad (9.2)$$

Then we would still like to be able to conclude that *Evil(John)*, because we know that John is a king (given) and John is greedy (because everyone is greedy). What we need for this to work is to find a substitution for both the variables in the implication sentence and the variables in the sentences that are in the knowledge base. In this case, applying the substitution $\{x/John, y/John\}$ to the implication premises *King*(*x*) and *Greedy*(*x*) and the knowledge-base sentences *King*(*John*) and *Greedy*(*y*) will make them identical. Thus, we can infer the consequent of the implication.

This inference process can be captured as a single inference rule that we call **Generalized Modus Ponens**.² For atomic sentences p_i , p_i' , and q , where there is a substitution θ such that

Generalized Modus Ponens

² Generalized Modus Ponens is more general than Modus Ponens (page 241) in the sense that the known facts and the premise of the implication need match only up to a substitution, rather than exactly. On the other hand, Modus Ponens allows any sentence α as the premise, rather than just a conjunction of atomic sentences.

$$\frac{\text{SUBST}(\theta, p_i') = \text{SUBST}(\theta, p_i), \text{ for all } i, \quad p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}.$$

There are $n + 1$ premises to this rule: the n atomic sentences p_i' and the one implication. The conclusion is the result of applying the substitution θ to the consequent q . For our example:

$$\begin{array}{ll} p_1' \text{ is } \textit{King}(\textit{John}) & p_1 \text{ is } \textit{King}(x) \\ p_2' \text{ is } \textit{Greedy}(y) & p_2 \text{ is } \textit{Greedy}(x) \\ \theta \text{ is } \{x/\textit{John}, y/\textit{John}\} & q \text{ is } \textit{Evil}(x) \\ \text{SUBST}(\theta, q) \text{ is } \textit{Evil}(\textit{John}). \end{array}$$

It is easy to show that Generalized Modus Ponens is a sound inference rule. First, we observe that, for any sentence p (whose variables are assumed to be universally quantified) and for any substitution θ ,

$$p \models \text{SUBST}(\theta, p)$$

is true by Universal Instantiation. It is true in particular for a θ that satisfies the conditions of the Generalized Modus Ponens rule. Thus, from $p_1', \dots, p_n'$ we can infer

$$\text{SUBST}(\theta, p_1') \wedge \dots \wedge \text{SUBST}(\theta, p_n')$$

and from the implication $p_1 \wedge \dots \wedge p_n \Rightarrow q$ we can infer

$$\text{SUBST}(\theta, p_1) \wedge \dots \wedge \text{SUBST}(\theta, p_n) \Rightarrow \text{SUBST}(\theta, q).$$

Now, θ in Generalized Modus Ponens is defined so that $\text{SUBST}(\theta, p_i') = \text{SUBST}(\theta, p_i)$, for all i ; therefore the first of these two sentences matches the premise of the second exactly. Hence, $\text{SUBST}(\theta, q)$ follows by Modus Ponens.

Generalized Modus Ponens is a **lifted** version of Modus Ponens—it raises Modus Ponens from ground (variable-free) propositional logic to first-order logic. We will see in the rest of this chapter that we can develop lifted versions of the forward chaining, backward chaining, and resolution algorithms introduced in Chapter 7. The key advantage of lifted inference rules over propositionalization is that they make only those substitutions that are required to allow particular inferences to proceed. Lifting

9.2.1 Unification

Lifted inference rules require finding substitutions that make different logical expressions look identical. This process is called **unification** and is a key component of all first-order inference algorithms. The UNIFY algorithm takes two sentences and returns a **unifier** for them (a substitution) if one exists: Unification
Unifier

$$\text{UNIFY}(p, q) = \theta \text{ where } \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q).$$

Let us look at some examples of how UNIFY should behave. Suppose we have a query $\text{AskVars}(\text{Knows}(\textit{John}, x))$: whom does John know? Answers to this query can be found by finding all sentences in the knowledge base that unify with $\text{Knows}(\textit{John}, x)$. Here are the results of unification with four different sentences that might be in the knowledge base:

$$\begin{array}{l} \text{UNIFY}(\text{Knows}(\textit{John}, x), \text{Knows}(\textit{John}, \textit{Jane})) = \{x/\textit{Jane}\} \\ \text{UNIFY}(\text{Knows}(\textit{John}, x), \text{Knows}(y, \textit{Bill})) = \{x/\textit{Bill}, y/\textit{John}\} \\ \text{UNIFY}(\text{Knows}(\textit{John}, x), \text{Knows}(y, \textit{Mother}(y))) = \{y/\textit{John}, x/\textit{Mother}(\textit{John})\} \\ \text{UNIFY}(\text{Knows}(\textit{John}, x), \text{Knows}(x, \textit{Elizabeth})) = \textit{failure}. \end{array}$$

Standardizing apart

The last unification fails because x cannot take on the values *John* and *Elizabeth* at the same time. Now, remember that $\text{Knows}(x, \text{Elizabeth})$ means “Everyone knows Elizabeth,” so we *should* be able to infer that John knows Elizabeth. The problem arises only because the two sentences happen to use the same variable name, x . The problem can be avoided by **standardizing apart** one of the two sentences being unified, which means renaming its variables to avoid name clashes. For example, we can rename x in $\text{Knows}(x, \text{Elizabeth})$ to x_{17} (a new variable name) without changing its meaning. Now the unification will work:

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x_{17}, \text{Elizabeth})) = \{x/\text{Elizabeth}, x_{17}/\text{John}\}.$$

Exercise 9.STAN delves further into the need for standardizing apart.

There is one more complication: we said that UNIFY should return a substitution that makes the two arguments look the same. But there could be more than one such unifier. For example, $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, z))$ could return $\{y/\text{John}, x/z\}$ or could return $\{y/\text{John}, x/\text{John}, z/\text{John}\}$. The first unifier gives $\text{Knows}(\text{John}, z)$ as the result of unification, whereas the second gives $\text{Knows}(\text{John}, \text{John})$. The second result could be obtained from the first by an additional substitution $\{z/\text{John}\}$; we say that the first unifier is *more general* than the second, because it places fewer restrictions on the values of the variables.

Most general unifier (MGU)

Every unifiable pair of expressions has a single **most general unifier** (MGU) that is unique up to renaming and substitution of variables. For example, $\{x/\text{John}\}$ and $\{y/\text{John}\}$ are considered equivalent, as are $\{x/\text{John}, y/\text{John}\}$ and $\{x/\text{John}, y/x\}$.

An algorithm for computing most general unifiers is shown in Figure 9.1. The process is simple: recursively explore the two expressions simultaneously “side by side,” building up a unifier along the way, but failing if two corresponding points in the structures do not match. There is one expensive step: when matching a variable against a complex term, one must check whether the variable itself occurs inside the term; if it does, the match fails because no consistent unifier can be constructed. For example, $S(x)$ can’t unify with $S(S(x))$. This so-called **occur check** makes the complexity of the entire algorithm quadratic in the size of the expressions being unified. Some systems, including many logic programming systems, simply omit the occur check and put the onus on the user to avoid making unsound inferences as a result. Other systems use more complex unification algorithms with linear-time complexity.

Occur check

9.2.2 Storage and retrieval

Underlying the TELL, ASK, and ASKVARs functions used to inform and interrogate a knowledge base are the more primitive STORE and FETCH functions. STORE(s) stores a sentence s into the knowledge base and FETCH(q) returns all unifiers such that the query q unifies with some sentence in the knowledge base. The problem we used to illustrate unification—finding all facts that unify with $\text{Knows}(\text{John}, x)$ —is an instance of FETCHing.

The simplest way to implement STORE and FETCH is to keep all the facts in one long list and unify each query against every element of the list. Such a process is inefficient, but it works. The remainder of this section outlines ways to make retrieval more efficient.

We can make FETCH more efficient by ensuring that unifications are attempted only with sentences that have *some* chance of unifying. For example, there is no point in trying to unify $\text{Knows}(\text{John}, x)$ with $\text{Brother}(\text{Richard}, \text{John})$. We can avoid such unifications by **indexing** the facts in the knowledge base. A simple scheme called **predicate indexing** puts all the

Indexing

Predicate indexing

```

function UNIFY( $x, y, \theta = \text{empty}$ ) returns a substitution to make  $x$  and  $y$  identical, or failure
  if  $\theta = \text{failure}$  then return failure
  else if  $x = y$  then return  $\theta$ 
  else if VARIABLE?( $x$ ) then return UNIFY-VAR( $x, y, \theta$ )
  else if VARIABLE?( $y$ ) then return UNIFY-VAR( $y, x, \theta$ )
  else if COMPOUND?( $x$ ) and COMPOUND?( $y$ ) then
    return UNIFY(ARGS( $x$ ), ARGS( $y$ ), UNIFY(OP( $x$ ), OP( $y$ ),  $\theta$ ))
  else if LIST?( $x$ ) and LIST?( $y$ ) then
    return UNIFY(REST( $x$ ), REST( $y$ ), UNIFY(FIRST( $x$ ), FIRST( $y$ ),  $\theta$ ))
  else return failure

function UNIFY-VAR( $var, x, \theta$ ) returns a substitution
  if  $\{var/val\} \in \theta$  for some  $val$  then return UNIFY( $val, x, \theta$ )
  else if  $\{x/val\} \in \theta$  for some  $val$  then return UNIFY( $var, val, \theta$ )
  else if OCCUR-CHECK?( $var, x$ ) then return failure
  else return add  $\{var/x\}$  to  $\theta$ 

```

Figure 9.1 The unification algorithm. The arguments x and y can be any expression: a constant or variable, or a compound expression such as a complex sentence or term, or a list of expressions. The argument θ is a substitution, initially the empty substitution, but with $\{var/val\}$ pairs added to it as we recurse through the inputs, comparing the expressions element by element. In a compound expression such as $F(A, B)$, OP(x) field picks out the function symbol F and ARGS(x) field picks out the argument list (A, B) .

Knows facts in one bucket and all the *Brother* facts in another. The buckets can be stored in a hash table for efficient access.

Predicate indexing is useful when there are many predicate symbols but only a few clauses for each symbol. Sometimes, however, a predicate has many clauses. For example, suppose that the tax authorities want to keep track of who employs whom, using a predicate *Employs*(x, y). This would be a very large bucket with perhaps millions of employers and tens of millions of employees. Answering a query such as *Employs*($x, Richard$) with predicate indexing would require scanning the entire bucket.

For this particular query, it would help if facts were indexed both by predicate and by second argument, perhaps using a combined hash table key. Then we could simply construct the key from the query and retrieve exactly those facts that unify with the query. For other queries, such as *Employs*(*IBM*, y), we would need to have indexed the facts by combining the predicate with the first argument. Therefore, facts can be stored under multiple index keys, rendering them instantly accessible to various queries that they might unify with.

Given a sentence to be stored, it is possible to construct indices for *all possible* queries that unify with it. For the fact *Employs*(*IBM*, *Richard*), the queries are

<i>Employs</i> (<i>IBM</i> , <i>Richard</i>)	Does IBM employ Richard?
<i>Employs</i> (x , <i>Richard</i>)	Who employs Richard?
<i>Employs</i> (<i>IBM</i> , y)	Whom does IBM employ?
<i>Employs</i> (x , y)	Who employs whom?

These queries form a **subsumption lattice**, as shown in Figure 9.2(a). The lattice has some [Subsumption lattice](#)

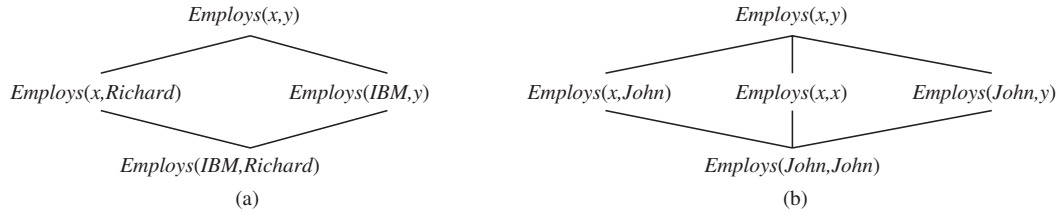


Figure 9.2 (a) The subsumption lattice whose lowest node is $Employs(IBM, Richard)$. (b) The subsumption lattice for the sentence $Employs(John, John)$.

interesting properties. The child of any node in the lattice is obtained from its parent by a single substitution; and the “highest” common descendant of any two nodes is the result of applying their most general unifier. A sentence with repeated constants has a slightly different lattice, as shown in Figure 9.2(b). Although function symbols are not shown in the figure, they too can be incorporated into the lattice structure.

For predicates with a small number of arguments, it is a good tradeoff to create an index for every point in the subsumption lattice. That requires a little more work at storage time, but speeds up retrieval time. However, for a predicate with n arguments, the lattice contains $O(2^n)$ nodes. If function symbols are allowed, the number of nodes is also exponential in the size of the terms in the sentence to be stored. This can lead to a huge number of indices.

We have to somehow limit the indices to ones that are likely to be used frequently in queries; otherwise we will waste more time in creating the indices than we save by having them. We could adopt a fixed policy, such as maintaining indices only on keys composed of a predicate plus a single argument. Or we could learn an adaptive policy that creates indices to meet the demands of the kinds of queries being asked. For commercial databases where facts number in the billions, the problem has been the subject of intensive study, technology development, and continual optimization.

9.3 Forward Chaining

In Section 7.5 we showed a forward-chaining algorithm for knowledge bases of propositional definite clauses. Here we expand that idea to cover first-order definite clauses.

Of course there are some logical sentences that cannot be stated as a definite clause, and thus cannot be handled by this approach. But rules of the form *Antecedent* $\Rightarrow$ *Consequent* are sufficient to cover a wide variety of interesting real-world systems.

9.3.1 First-order definite clauses

First-order definite clauses are disjunctions of literals of which *exactly one is positive*. That means a definite clause is either atomic, or is an implication whose antecedent is a conjunction of positive literals and whose consequent is a single positive literal. Existential quantifiers are not allowed, and universal quantifiers are left implicit: if you see an x in a definite clause, that means there is an implicit $\forall x$ quantifier. A typical first-order definite clause looks like this:

$$King(x) \wedge Greedy(x) \Rightarrow Evil(x),$$

but the literals $King(John)$ and $Greedy(y)$ also count as definite clauses. First-order liter-

als can include variables, so *Greedy*(y) is interpreted as “everyone is greedy” (the universal quantifier is implicit).

Let us put definite clauses to work in representing the following problem:

The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.

First, we will represent these facts as first-order definite clauses:

“... it is a crime for an American to sell weapons to hostile nations”:

$$\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x). \quad (9.3)$$

“Nono ... has some missiles.” The sentence $\exists x \text{ Owns}(\text{Nono}, x) \wedge \text{Missile}(x)$ is transformed into two definite clauses by Existential Instantiation, introducing a new constant M_1 :

$$\text{Owns}(\text{Nono}, M_1) \quad (9.4)$$

$$\text{Missile}(M_1) \quad (9.5)$$

“All of its missiles were sold to it by Colonel West”:

$$\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono}). \quad (9.6)$$

We will also need to know that missiles are weapons:

$$\text{Missile}(x) \Rightarrow \text{Weapon}(x) \quad (9.7)$$

and we must know that an enemy of America counts as “hostile”:

$$\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x). \quad (9.8)$$

“West, who is American ...”:

$$\text{American}(\text{West}). \quad (9.9)$$

“The country Nono, an enemy of America ...”:

$$\text{Enemy}(\text{Nono}, \text{America}). \quad (9.10)$$

This knowledge base happens to be a **Datalog** knowledge base: Datalog is a language consisting of first-order definite clauses with no function symbols. Datalog gets its name because it can represent the type of statements typically made in relational databases. The absence of function symbols makes inference much easier. Datalog

9.3.2 A simple forward-chaining algorithm

Figure 9.3 shows a simple forward chaining inference algorithm. Starting from the known facts, it triggers all the rules whose premises are satisfied, adding their conclusions to the known facts. The process repeats until the query is answered (assuming that just one answer is required) or no new facts are added. Notice that a fact is not “new” if it is just a **renaming** of a known fact—a sentence is a renaming of another if they are identical except for the names of the variables. For example, *Likes*(x, *IceCream*) and *Likes*(y, *IceCream*) are renamings of each other. They both mean the same thing: “Everyone likes ice cream.” Renaming

We use our crime problem to illustrate FOL-FC-ASK. The implication sentences available for chaining are (9.3), (9.6), (9.7), and (9.8). Two iterations are required:

```

function FOL-FC-ASK( $KB, \alpha$ ) returns a substitution or false
  inputs:  $KB$ , the knowledge base, a set of first-order definite clauses
            $\alpha$ , the query, an atomic sentence

  while true do
     $new \leftarrow \{\}$  // The set of new sentences inferred on each iteration
    for each rule in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$ 
      for each  $\theta$  such that  $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$ 
        for some  $p'_1, \dots, p'_n$  in  $KB$ 
         $q' \leftarrow \text{SUBST}(\theta, q)$ 
        if  $q'$  does not unify with some sentence already in  $KB$  or  $new$  then
          add  $q'$  to  $new$ 
           $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
          if  $\phi$  is not failure then return  $\phi$ 
    if  $new = \{\}$  then return false
    add  $new$  to  $KB$ 

```

Figure 9.3 A conceptually straightforward, but inefficient, forward-chaining algorithm. On each iteration, it adds to KB all the atomic sentences that can be inferred in one step from the implication sentences and the atomic sentences already in KB . The function `STANDARDIZE-VARIABLES` replaces all variables in its arguments with new ones that have not been used before.

- On the first iteration, rule (9.3) has unsatisfied premises.
 Rule (9.6) is satisfied with $\{x/M_1\}$, and $\text{Sells}(\text{West}, M_1, \text{Nono})$ is added.
 Rule (9.7) is satisfied with $\{x/M_1\}$, and $\text{Weapon}(M_1)$ is added.
 Rule (9.8) is satisfied with $\{x/\text{Nono}\}$, and $\text{Hostile}(\text{Nono})$ is added.
- On the second iteration, rule (9.3) is satisfied with $\{x/\text{West}, y/M_1, z/\text{Nono}\}$, and the inference $\text{Criminal}(\text{West})$ is added.

Figure 9.4 shows the proof tree that is generated. Notice that no new inferences are possible at this point because every sentence that could be concluded by forward chaining is already contained explicitly in the KB. Such a knowledge base is called a **fixed point** of the inference process. Fixed points reached by forward chaining with first-order definite clauses are similar to those for propositional forward chaining (page 249); the principal difference is that a first-order fixed point can include universally quantified atomic sentences.

FOL-FC-ASK is easy to analyze. First, it is **sound**, because every inference is just an application of Generalized Modus Ponens, which is sound. Second, it is **complete** for definite clause knowledge bases; that is, it answers every query whose answers are entailed by any knowledge base of definite clauses.

For Datalog knowledge bases, which contain no function symbols, the proof of completeness is fairly easy. We begin by counting the number of possible facts that can be added, which determines the maximum number of iterations. Let k be the maximum **arity** (number of arguments) of any predicate, p be the number of predicates, and n be the number of constant symbols. Clearly, there can be no more than pn^k distinct ground facts, so after this many iterations the algorithm must have reached a fixed point. Then we can make an argument very

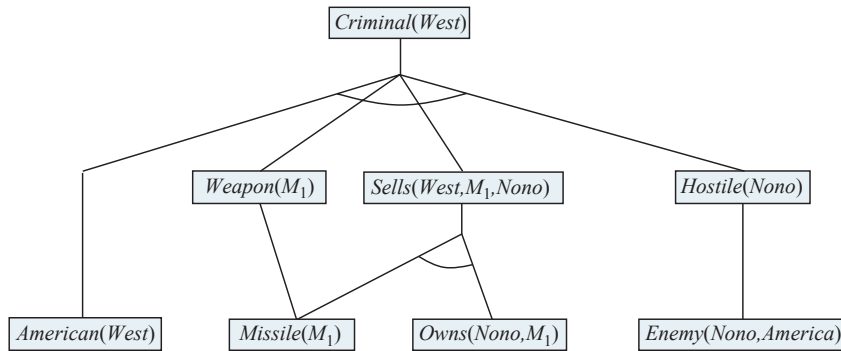


Figure 9.4 The proof tree generated by forward chaining on the crime example. The initial facts appear at the bottom level, facts inferred on the first iteration in the middle level, and facts inferred on the second iteration at the top level.

similar to the proof of completeness for propositional forward chaining. (See page 249.) The details of how to make the transition from propositional to first-order completeness are given for the resolution algorithm in Section 9.5.

For general definite clauses with function symbols, FOL-FC-ASK can generate infinitely many new facts, so we need to be more careful. For the case in which an answer to the query sentence q is entailed, we must appeal to Herbrand's theorem (page 300) to establish that the algorithm will find a proof. (See Section 9.5 for the resolution case.) If the query has no answer, the algorithm could fail to terminate in some cases. For example, if the knowledge base includes the Peano axioms

$$\begin{aligned} & \text{NatNum}(0) \\ & \forall n \text{ NatNum}(n) \Rightarrow \text{NatNum}(S(n)), \end{aligned}$$

then forward chaining adds $\text{NatNum}(S(0))$, $\text{NatNum}(S(S(0)))$, $\text{NatNum}(S(S(S(0))))$, and so on. This problem is unavoidable in general. As with general first-order logic, entailment with definite clauses is semidecidable.

9.3.3 Efficient forward chaining

The forward-chaining algorithm in Figure 9.3 is designed for ease of understanding, not efficiency. There are three sources of inefficiency. First, the inner loop of the algorithm tries to match every rule against every fact in the knowledge base. Second, the algorithm rechecks every rule on every iteration, even if very few additions have been made to the knowledge base. Third, the algorithm can generate many facts that are irrelevant to the goal. We address each of these issues in turn.

Matching rules against known facts

The problem of matching the premise of a rule against the facts in the knowledge base might seem simple enough. For example, suppose we want to apply the rule

$$\text{Missile}(x) \Rightarrow \text{Weapon}(x).$$

Then we need to find all the facts that unify with $\text{Missile}(x)$; in a suitably indexed knowledge

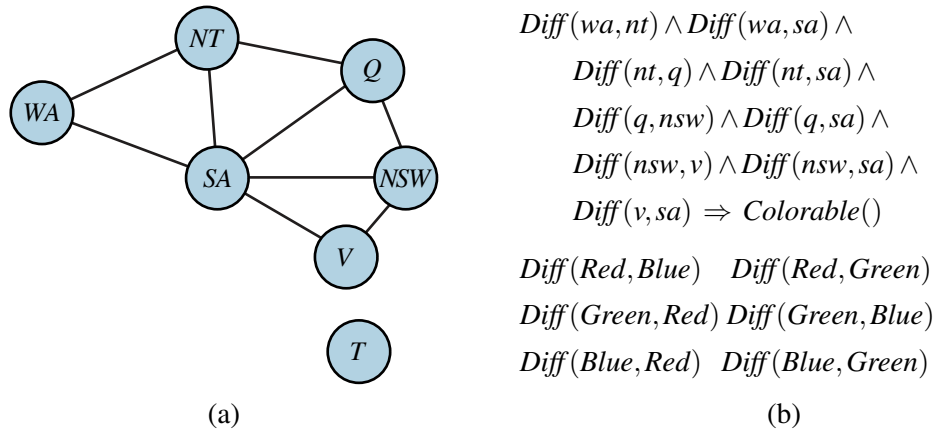


Figure 9.5 (a) Constraint graph for coloring the map of Australia. (b) The map-coloring CSP expressed as a single definite clause. Each map region is represented as a variable whose value can be one of the constants *Red*, *Green*, or *Blue* (which are declared *Diff*).

base, this can be done in constant time per fact. Now consider a rule such as

$$Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono).$$

Again, we can find all the objects owned by Nono in constant time per object; then, for each object, we could check whether it is a missile. However, if the knowledge base contains many objects owned by Nono and very few missiles, then it would be better to find all the missiles first and then check whether they are owned by Nono. This is the **conjunct ordering** problem: find an ordering to solve the conjuncts of the rule premise so that the total cost is minimized. It turns out that finding the optimal ordering is NP-hard, but good heuristics are available. For example, the **minimum-remaining-values** (MRV) heuristic used for CSPs in Chapter 5 would suggest ordering the conjuncts to look for missiles first if there are fewer missiles than there are objects owned by Nono.

The connection between this **pattern matching** and constraint satisfaction is actually very close. We can view each conjunct as a constraint on the variables that it contains—for example, *Missile*(*x*) is a unary constraint on *x*. Extending this idea, *we can express every finite-domain CSP as a single definite clause together with some associated ground facts*. Consider the map-coloring problem from Figure 5.1, shown again in Figure 9.5(a). An equivalent formulation as a single definite clause is given in Figure 9.5(b). Clearly, the conclusion *Colorable*() can be inferred only if the CSP has a solution. Because CSPs in general include 3-SAT problems as special cases, we can conclude that *matching a definite clause against a set of facts is NP-hard*.

It might seem rather depressing that forward chaining has an NP-hard matching problem in its inner loop. There are three ways to cheer ourselves up:

- We can remind ourselves that most rules in real-world knowledge bases are small and simple (like the rules in our crime example) rather than large and complex (like the CSP formulation in Figure 9.5). It is common in the database world to assume that both the sizes of rules and the arities of predicates are bounded by a constant and to worry only about **data complexity**—that is, the complexity of inference as a function

of the number of ground facts in the knowledge base. It is easy to show that the data complexity of forward chaining is polynomial, not exponential.

- We can consider subclasses of rules for which matching is efficient. Essentially every Datalog clause can be viewed as defining a CSP, so matching will be tractable just when the corresponding CSP is tractable. Chapter 5 describes several tractable families of CSPs. For example, if the constraint graph (the graph whose nodes are variables and whose links are constraints) forms a tree, then the CSP can be solved in linear time. Exactly the same result holds for rule matching. For instance, if we remove South Australia from the map in Figure 9.5, the resulting clause is

$$\text{Diff}(wa, nt) \wedge \text{Diff}(nt, q) \wedge \text{Diff}(q, nsw) \wedge \text{Diff}(nsw, v) \Rightarrow \text{Colorable}()$$

which corresponds to the reduced CSP shown in Figure 5.12 on page 185. Algorithms for solving tree-structured CSPs can be applied directly to the problem of rule matching.

- We can try to eliminate redundant rule-matching attempts in the forward-chaining algorithm, as described next.

Incremental forward chaining

When we showed how forward chaining works on the crime example, we cheated. In particular, we omitted some of the rule matching done by the algorithm shown in Figure 9.3. For example, on the second iteration, the rule

$$\text{Missile}(x) \Rightarrow \text{Weapon}(x)$$

matches against $\text{Missile}(M_1)$ (again), and of course the conclusion $\text{Weapon}(M_1)$ is already known so nothing happens. Such redundant rule matching can be avoided if we make the following observation: *Every new fact inferred on iteration t must be derived from at least one new fact inferred on iteration $t - 1$.* This is true because any inference that does not require a new fact from iteration $t - 1$ could have been done at iteration $t - 1$ already. 

This observation leads naturally to an incremental forward-chaining algorithm where, at iteration t , we check a rule only if its premise includes a conjunct p_i that unifies with a fact p'_i newly inferred at iteration $t - 1$. The rule-matching step then fixes p_i to match with p'_i , but allows the other conjuncts of the rule to match with facts from any previous iteration. This algorithm generates exactly the same facts at each iteration as the algorithm in Figure 9.3, but is much more efficient.

With suitable indexing, it is easy to identify all the rules that can be triggered by any given fact, and many real systems operate in an “update” mode wherein forward chaining occurs in response to every TELL. Inferences cascade through the set of rules until the fixed point is reached, and then the process begins again for the next new fact.

Typically, only a small fraction of the rules in the knowledge base are actually triggered by the addition of a given fact. This means that a great deal of redundant work is done in repeatedly constructing partial matches that have some unsatisfied premises. Our crime example is rather too small to show this effectively, but notice that a partial match is constructed on the first iteration between the rule

$$\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$$

and the fact $\text{American}(\text{West})$. This partial match is then discarded and rebuilt on the second iteration (when the rule succeeds). It would be better to retain and gradually complete the partial matches as new facts arrive, rather than discarding them.

Rete algorithm

The **Rete algorithm**³ was the first to address this problem. The algorithm preprocesses the set of rules in the knowledge base to construct a dataflow network in which each node is a literal from a rule premise. Variable bindings flow through the network and are filtered out when they fail to match a literal. If two literals in a rule share a variable—for example, $Sells(x, y, z) \wedge Hostile(z)$ in the crime example—then the bindings from each literal are filtered through an equality node. A variable binding reaching a node for an n -ary literal such as $Sells(x, y, z)$ might have to wait for bindings for the other variables to be established before the process can continue. At any given point, the state of a Rete network captures all the partial matches of the rules, avoiding a great deal of recomputation.

Production system

Rete networks, and various improvements thereon, have been a key component of so-called **production systems**, which were among the earliest forward-chaining systems in widespread use.⁴ The XCON system (originally called R1; McDermott, 1982) was built with a production-system architecture. XCON contained several thousand rules for designing configurations of computer components for customers of the Digital Equipment Corporation. It was one of the first clear commercial successes in the emerging field of expert systems. Many other similar systems have been built with the same underlying technology, which has been implemented in the general-purpose language OPS-5.

Cognitive architectures

Production systems are also popular in **cognitive architectures**—that is, models of human reasoning—such as ACT (Anderson, 1983) and SOAR (Laird *et al.*, 1987). In such systems, the “working memory” of the system models human short-term memory, and the productions are part of long-term memory. On each cycle of operation, productions are matched against the working memory of facts. A production whose conditions are satisfied can add or delete facts in working memory. In contrast to the typical situation in databases, production systems often have many rules and relatively few facts. With suitably optimized matching technology, systems can operate in real time with tens of millions of rules.

Irrelevant facts

Another source of inefficiency is that forward chaining makes all allowable inferences based on the known facts, *even if they are irrelevant to the goal*. In our crime example, there were no rules capable of drawing irrelevant conclusions. But if there had been many rules describing the eating habits of Americans or the components and prices of missiles, then FOL-FC-ASK would have generated irrelevant conclusions.

Deductive databases

One way to avoid drawing irrelevant conclusions is to use backward chaining, as described in Section 9.4. Another way is to restrict forward chaining to a selected subset of rules, as in PL-FC-ENTAILS? (page 249). A third approach has emerged in the field of **deductive databases**, which are large-scale databases, like relational databases, but which use forward chaining as the standard inference tool rather than SQL queries. The idea is to rewrite the rule set, using information from the goal, so that only relevant variable bindings—those belonging to a so-called **magic set**—are considered during forward inference. For example, if the goal is $Criminal(West)$, the rule that concludes $Criminal(x)$ will be rewritten to include an extra conjunct that constrains the value of x :

Magic set

$$Magic(x) \wedge American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x).$$

³ Rete is Latin for *net*. It rhymes with *treaty*.

⁴ The word **production** in **production systems** denotes a condition–action rule.

The fact *Magic(West)* is also added to the KB. In this way, even if the knowledge base contains facts about millions of Americans, only Colonel West will be considered during the forward inference process. The complete process for defining magic sets and rewriting the knowledge base is too complex to go into here, but the basic idea is to perform a sort of “generic” backward inference from the goal in order to work out which variable bindings need to be constrained. The magic sets approach can therefore be thought of as a kind of hybrid between forward inference and backward preprocessing.

9.4 Backward Chaining

The second major family of logical inference algorithms uses **backward chaining** over definite clauses. These algorithms work backward from the goal, chaining through rules to find known facts that support the proof.

9.4.1 A backward-chaining algorithm

Figure 9.6 shows a backward-chaining algorithm for definite clauses. FOL-BC-ASK(*KB*, *goal*) will be proved if the knowledge base contains a rule of the form $lhs \Rightarrow goal$, where *lhs* (left-hand side) is a list of conjuncts. An atomic fact like *American(West)* is considered as a clause whose *lhs* is the empty list. Now a query that contains variables might be proved in multiple ways. For example, the query *Person(x)* could be proved with the substitution $\{x/John\}$ as well as with $\{x/Richard\}$. So we implement FOL-BC-ASK as a generator—a function that returns multiple times, each time giving one possible result (see Appendix B).

Backward chaining is a kind of AND/OR search—the OR part because the goal query can be proved by any rule in the knowledge base, and the AND part because all the conjuncts in the *lhs* of a clause must be proved. FOL-BC-OR works by fetching all clauses that might

```
function FOL-BC-ASK(KB, query) returns a generator of substitutions
  return FOL-BC-OR(KB, query, { })
```

```
function FOL-BC-OR(KB, goal,  $\theta$ ) returns a substitution
  for each rule in FETCH-RULES-FOR-GOAL(KB, goal) do
    ( $lhs \Rightarrow rhs$ )  $\leftarrow$  STANDARDIZE-VARIABLES(rule)
    for each  $\theta'$  in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal,  $\theta$ )) do
      yield  $\theta'$ 
```

```
function FOL-BC-AND(KB, goals,  $\theta$ ) returns a substitution
  if  $\theta = failure$  then return
  else if LENGTH(goals) = 0 then yield  $\theta$ 
  else
     $first, rest \leftarrow$  FIRST(goals), REST(goals)
    for each  $\theta'$  in FOL-BC-OR(KB, SUBST( $\theta$ , first),  $\theta$ ) do
      for each  $\theta''$  in FOL-BC-AND(KB, rest,  $\theta'$ ) do
        yield  $\theta''$ 
```

Figure 9.6 A simple backward-chaining algorithm for first-order knowledge bases.

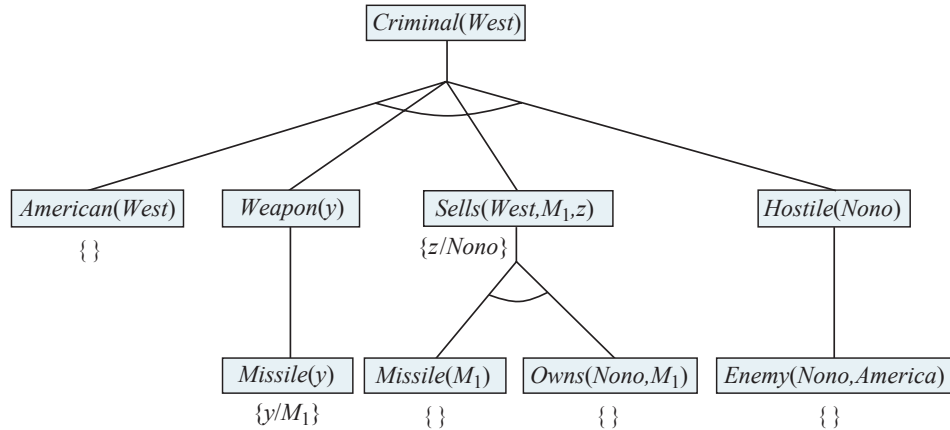


Figure 9.7 Proof tree constructed by backward chaining to prove that West is a criminal. The tree should be read depth first, left to right. To prove *Criminal(West)*, we have to prove the four conjuncts below it. Some of these are in the knowledge base, and others require further backward chaining. Bindings for each successful unification are shown next to the corresponding subgoal. Note that once one subgoal in a conjunction succeeds, its substitution is applied to subsequent subgoals. Thus, by the time FOL-BC-ASK gets to the last conjunct, originally *Hostile(z)*, *z* is already bound to *Nono*.

unify with the goal, standardizing the variables in the clause to be brand-new variables, and then, if the *rhs* of the clause does indeed unify with the goal, proving every conjunct in the *lhs*, using FOL-BC-AND. That function works by proving each of the conjuncts in turn, keeping track of the accumulated substitution as it goes. Figure 9.7 is the proof tree for deriving *Criminal(West)* from sentences (9.3) through (9.10).

Backward chaining, as we have written it, is clearly a depth-first search algorithm. This means that its space requirements are linear in the size of the proof. It also means that backward chaining (unlike forward chaining) suffers from problems with repeated states and incompleteness. Despite these limitations, backward chaining has proven to be popular and effective in logic programming languages.

9.4.2 Logic programming

Logic programming is a technology that comes close to embodying the declarative ideal described in Chapter 7: that systems should be constructed by expressing knowledge in a formal language and that problems should be solved by running inference processes on that knowledge. The ideal is summed up in Robert Kowalski's equation,

$$\text{Algorithm} = \text{Logic} + \text{Control}.$$

Prolog

Prolog is the most widely used logic programming language. It is used primarily as a rapid-prototyping language and for symbol-manipulation tasks such as writing compilers (Van Roy, 1990) and parsing natural language (Pereira and Warren, 1980). Many expert systems have been written in Prolog for legal, medical, financial, and other domains.

Prolog programs are sets of definite clauses written in a notation somewhat different from standard first-order logic. Prolog uses uppercase letters for variables and lowercase for constants—the opposite of our convention for logic. Commas separate conjuncts in a clause,

and the clause is written “backwards” from what we are used to; instead of $A \wedge B \Rightarrow C$ in Prolog we have $C :- A, B$. Here is a typical example:

```
criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).
```

In Prolog the notation $[E|L]$ denotes a list whose first element is E and whose rest is L . Here is a Prolog program for $\text{append}(X,Y,Z)$, which succeeds if list Z is the result of appending lists X and Y :

```
append([],Y,Y).
append([A|X],Y,[A|Z]) :- append(X,Y,Z).
```

In English, we can read these clauses as (1) appending the empty list and the list Y produces the same list Y , and (2) $[A|Z]$ is the result of appending $[A|X]$ and Y , provided that Z is the result of appending X and Y . In most high-level languages we can write a similar recursive function that describes how to append two lists. The Prolog definition is actually more powerful, however, because it describes a *relation* that holds among three arguments, rather than a *function* computed from two arguments. For example, we can ask the query $\text{append}(X,Y,[1,2,3])$: what two lists can be appended to give $[1,2,3]$? Prolog gives us back the solutions

```
X=[]      Y=[1,2,3];
X=[1]     Y=[2,3];
X=[1,2]   Y=[3];
X=[1,2,3] Y=[]
```

The execution of Prolog programs is done through depth-first backward chaining, where clauses are tried in the order in which they are written in the knowledge base. Prolog’s design represents a compromise between declarativeness and execution efficiency. Some aspects of Prolog fall outside standard logical inference:

- Prolog uses the database semantics of Section 8.2.8 rather than first-order semantics, and this is apparent in its treatment of equality and negation (see Section 9.4.4).
- There is a set of built-in functions for arithmetic. Literals using these function symbols are “proved” by executing code rather than doing further inference. For example, the goal “ X is $4+3$ ” succeeds with X bound to 7. On the other hand, the goal “5 is $X+Y$ ” fails, because the built-in functions do not do arbitrary equation solving.
- There are built-in predicates that have side effects when executed. These include input–output predicates and the `assert/retract` predicates for modifying the knowledge base. Such predicates have no counterpart in logic and can produce confusing results—for example, if facts are asserted in a branch of the proof tree that eventually fails.
- The **occur check** is omitted from Prolog’s unification algorithm. This means that some unsound inferences can be made; these are almost never a problem in practice.
- Prolog uses depth-first backward-chaining search with no checks for infinite recursion. This makes for a usable programming language that is very fast when used properly, but it means that some programs that look like valid logic will fail to terminate.

9.4.3 Redundant inference and infinite loops

We now turn to the Achilles heel of Prolog: the mismatch between depth-first search and search trees that include repeated states and infinite paths. Consider the following logic program that decides if a path exists between two points on a directed graph:

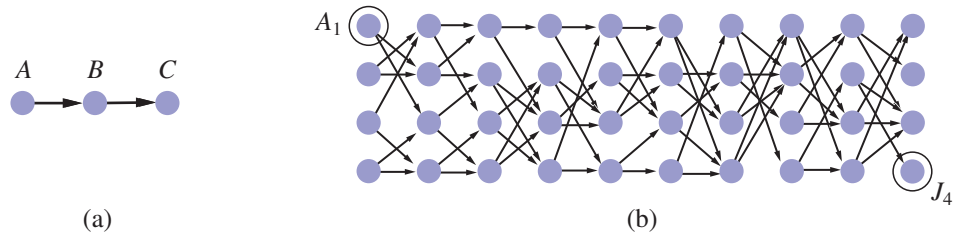


Figure 9.8 (a) Finding a path from A to C can lead Prolog into an infinite loop. (b) A graph in which each node is connected to two random successors in the next layer. Finding a path from A₁ to J₄ requires 877 inferences.

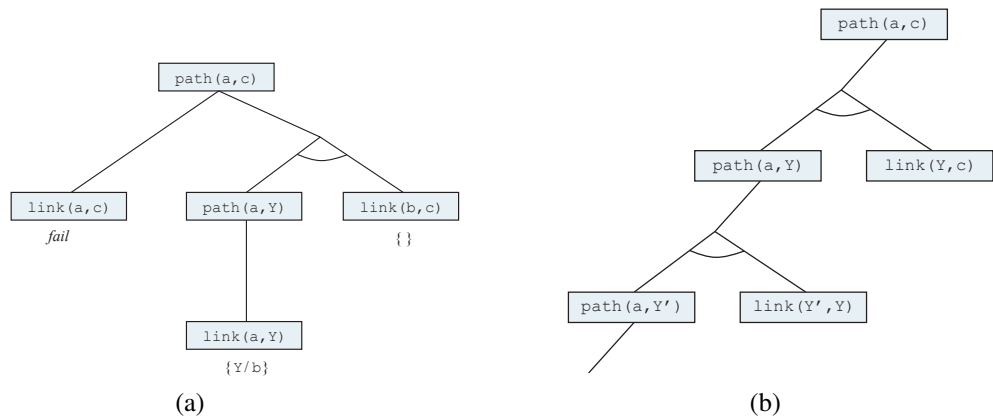


Figure 9.9 (a) Proof that a path exists from A to C. (b) Infinite proof tree generated when the clauses are in the “wrong” order.

```
path(X,Z) :- link(X,Z).
path(X,Z) :- path(X,Y), link(Y,Z).
```

A simple three-node graph, described by the facts `link(a,b)` and `link(b,c)`, is shown in Figure 9.8(a). With this program, the query `path(a,c)` generates the proof tree shown in Figure 9.9(a). On the other hand, if we put the two clauses in the order

```
path(X,Z) :- path(X,Y), link(Y,Z).
path(X,Z) :- link(X,Z).
```

then Prolog follows the infinite path shown in Figure 9.9(b). Prolog is therefore **incomplete** as a theorem prover for definite clauses—even for Datalog programs, as this example shows—because, for some knowledge bases, it fails to prove sentences that are entailed. Notice that forward chaining does not suffer from this problem: once `path(a,b)`, `path(b,c)`, and `path(a,c)` are inferred, forward chaining halts.

Depth-first backward chaining also has problems with redundant computations. For example, when finding a path from A₁ to J₄ in Figure 9.8(b), Prolog performs 877 inferences, most of which involve finding all possible paths to nodes from which the goal is unreachable. This is similar to the repeated-state problem discussed in Chapter 3. The total amount of

inference can be exponential in the number of ground facts that are generated. If we apply forward chaining instead, at most n^2 $\text{path}(X, Y)$ facts can be generated linking n nodes. For the problem in Figure 9.8(b), only 62 inferences are needed.

Forward chaining on graph search problems is an example of **dynamic programming**, in which the solutions to subproblems are constructed incrementally from those of smaller subproblems and are cached to avoid recomputation. We can obtain a similar effect in a backward chaining system, except that here we are breaking down large goals into smaller ones, rather than building them up.

Dynamic
programming

Either way, storing intermediate results to avoid duplication is key. This is the approach taken by **tabled logic programming** systems, which use efficient storage and retrieval mechanisms. Tabled logic programming combines the goal-directedness of backward chaining with the dynamic-programming efficiency of forward chaining. It is also complete for Datalog knowledge bases, which means that the programmer need worry less about infinite loops. (It is still possible to get an infinite loop with predicates like $\text{father}(X, Y)$ that refer to a potentially unbounded number of objects.)

Tabled logic
programming

9.4.4 Database semantics of Prolog

Prolog uses database semantics, as discussed in Section 8.2.8. The unique names assumption says that every Prolog constant and every ground term refers to a distinct object, and the closed world assumption says that the only sentences that are true are those that are entailed by the knowledge base. There is no way to assert that a sentence is false in Prolog. This makes Prolog less expressive than first-order logic, but it is part of what makes Prolog more efficient and more concise. Consider the following assertions about some course offerings:

$$\text{Course}(CS, 101), \text{Course}(CS, 102), \text{Course}(CS, 106), \text{Course}(EE, 101). \quad (9.11)$$

Under the unique names assumption, CS and EE are different (as are 101, 102, and 106), so this means that there are four distinct courses. Under the closed-world assumption there are no other courses, so there are exactly four courses. But if these were assertions in FOL rather than in database semantics, then all we could say is that there are somewhere between one and infinity courses. That's because the assertions (in FOL) do not deny the possibility that other unmentioned courses are also offered, nor do they say that the courses mentioned are different from each other. If we wanted to translate Equation (9.11) into FOL, we would get the following sentence:

$$\begin{aligned} \text{Course}(d, n) \Leftrightarrow & (d = CS \wedge n = 101) \vee (d = CS \wedge n = 102) \\ & \vee (d = CS \wedge n = 106) \vee (d = EE \wedge n = 101). \end{aligned} \quad (9.12)$$

This is called the **completion** of Equation (9.11). It expresses in FOL the idea that there are at most four courses. To express in FOL the idea that there are at least four courses, we need to write the completion of the equality predicate:

Completion

$$\begin{aligned} x = y \Leftrightarrow & (x = CS \wedge y = CS) \vee (x = EE \wedge y = EE) \vee (x = 101 \wedge y = 101) \\ & \vee (x = 102 \wedge y = 102) \vee (x = 106 \wedge y = 106). \end{aligned}$$

The completion is useful for understanding database semantics, but for practical purposes, if your problem can be described with database semantics, it is more efficient to reason with Prolog or some other database semantics system, rather than translating into FOL and reasoning with a full FOL theorem prover.

9.4.5 Constraint logic programming

In our discussion of forward chaining (Section 9.3), we showed how constraint satisfaction problems (CSPs) can be encoded as definite clauses. Standard Prolog solves such problems in exactly the same way as the backtracking algorithm given in Figure 5.5.

Because backtracking enumerates the domains of the variables, it works only for **finite-domain** CSPs. In Prolog terms, there must be a finite number of solutions for any goal with unbound variables. (For example, a map coloring problem in which each variable can take on one of four different colors.) Infinite-domain CSPs—for example, with integer- or real-valued variables—require quite different algorithms, such as bounds propagation or linear programming.

Consider the following example. We define `triangle(X,Y,Z)` as a predicate that holds if the three arguments are numbers that satisfy the triangle inequality:

```
triangle(X,Y,Z) :-
    X>0, Y>0, Z>0, X+Y>Z, Y+Z>X, X+Z>Y.
```

If we ask Prolog the query `triangle(3,4,5)`, it succeeds. On the other hand, if we ask `triangle(3,4,Z)`, no solution will be found, because the subgoal `Z>0` cannot be handled by Prolog; we can't compare an unbound value to 0.

Constraint logic programming

Constraint logic programming (CLP) allows variables to be *constrained* rather than *bound*. A CLP solution is the most specific set of constraints on the query variables that can be derived from the knowledge base. For example, the solution to the `triangle(3,4,Z)` query is the constraint $7 > Z > 1$. Standard logic programs are just a special case of CLP in which the solution constraints must be equality constraints—that is, bindings.

CLP systems incorporate various constraint-solving algorithms for the constraints allowed in the language. For example, a system that allows linear inequalities on real-valued variables might include a linear programming algorithm for solving those constraints. CLP systems also adopt a much more flexible approach to solving standard logic programming queries. For example, instead of depth-first, left-to-right backtracking, they might use any of the more efficient algorithms discussed in Chapter 5, including heuristic conjunct ordering, backjumping, cutset conditioning, and so on. CLP systems therefore combine elements of constraint satisfaction algorithms, logic programming, and deductive databases.

Several systems that allow the programmer more control over the search order for inference have been defined. The MRS language (Genesereth and Smith, 1981; Russell, 1985) allows the programmer to write **metarules** to determine which conjuncts are tried first. The user could write a rule saying that the goal with the fewest variables should be tried first or could write domain-specific rules for particular predicates.

Metarule

9.5 Resolution

The last of our three families of logical systems, and the only one that works for any knowledge base, not just definite clauses, is **resolution**. We saw on page 241 that propositional resolution is a complete inference procedure for propositional logic; in this section, we extend it to first-order logic.

9.5.1 Conjunctive normal form for first-order logic

The first step is to convert sentences to **conjunctive normal form** (CNF)—that is, a conjunction of clauses, where each clause is a disjunction of literals.⁵ In CNF, literals can contain variables, which are assumed to be universally quantified. For example, the sentence

$$\forall x, y, z \text{ American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$$

becomes, in CNF,

$$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x).$$

The key is that *Every sentence of first-order logic can be converted into an inferentially equivalent CNF sentence.* 

The procedure for conversion to CNF is similar to the propositional case, which we saw on page 244. The principal difference arises from the need to eliminate existential quantifiers. We illustrate the procedure by translating the sentence “Everyone who loves all animals is loved by someone,” or

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)].$$

The steps are as follows:

- **Eliminate implications:** Replace $P \Rightarrow Q$ with $\neg P \vee Q$. For our sample sentence, this needs to be done twice:

$$\begin{aligned} \forall x \neg[\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)] \\ \forall x \neg[\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)]. \end{aligned}$$

- **Move $\neg$ inwards:** In addition to the usual rules for negated connectives, we need rules for negated quantifiers. Thus, we have

$$\begin{aligned} \neg \forall x p & \text{ becomes } \exists x \neg p \\ \neg \exists x p & \text{ becomes } \forall x \neg p. \end{aligned}$$

Our sentence goes through the following transformations:

$$\begin{aligned} \forall x [\exists y \neg(\neg \text{Animal}(y) \vee \text{Loves}(x, y))] \vee [\exists y \text{ Loves}(y, x)]. \\ \forall x [\exists y \neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)]. \\ \forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)]. \end{aligned}$$

Notice how a universal quantifier ($\forall y$) in the premise of the implication has become an existential quantifier. The sentence now reads “Either there is some animal that x doesn’t love, or (if this is not the case) someone loves x .” Clearly, the meaning of the original sentence has been preserved.

- **Standardize variables:** For sentences like $(\exists x P(x)) \vee (\exists x Q(x))$ that use the same variable name twice, change the name of one of the variables. This avoids confusion later when we drop the quantifiers. Thus, we have

$$\forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{ Loves}(z, x)].$$

- **Skolemize: Skolemization** is the process of removing existential quantifiers by elimi- Skolemization

⁵ A clause can also be represented as an implication with a conjunction of atoms in the premise and a disjunction of atoms in the conclusion (Exercise 9.DISJ). This is called **implicative normal form** or **Kowalski form** (especially when written with a right-to-left implication symbol (Kowalski, 1979)) and is generally much easier to read than a disjunction with many negated literals.

nation. In the simple case, it is just like the Existential Instantiation rule of Section 9.1: translate $\exists x P(x)$ into $P(A)$, where A is a new constant. However, we can't apply Existential Instantiation to our sentence above because it doesn't match the pattern $\exists v \alpha$; only parts of the sentence match the pattern. If we blindly apply the rule to the two matching parts we get

$$\forall x [Animal(A) \wedge \neg Loves(x, A)] \vee Loves(B, x),$$

which has the wrong meaning entirely: it says that everyone either fails to love a particular animal A or is loved by some particular entity B . In fact, our original sentence allows each person to fail to love a different animal or to be loved by a different person. Thus, we want the Skolem entities to depend on x :

$$\forall x [Animal(F(x)) \wedge \neg Loves(x, F(x))] \vee Loves(G(x), x).$$

Skolem function

Here F and G are **Skolem functions**. The general rule is that the arguments of the Skolem function are all the universally quantified variables in whose scope the existential quantifier appears. As with Existential Instantiation, the Skolemized sentence is satisfiable exactly when the original sentence is satisfiable.

- **Drop universal quantifiers:** At this point, all remaining variables must be universally quantified. Therefore, we don't lose any information if we drop the quantifier:

$$[Animal(F(x)) \wedge \neg Loves(x, F(x))] \vee Loves(G(x), x).$$

- **Distribute $\vee$ over $\wedge$:**

$$[Animal(F(x)) \vee Loves(G(x), x)] \wedge [\neg Loves(x, F(x)) \vee Loves(G(x), x)].$$

This step may also require flattening out nested conjunctions and disjunctions.

The sentence is now in CNF and consists of two clauses. It is much more difficult to read than the original sentence with implications. (It may help to explain that the Skolem function $F(x)$ refers to the animal potentially unloved by x , whereas $G(x)$ refers to someone who might love x .) Fortunately, humans seldom need to look at CNF sentences—the translation process is easily automated.

9.5.2 The resolution inference rule

The resolution rule for first-order clauses is simply a lifted version of the propositional resolution rule given on page 244. Two clauses, which are assumed to be standardized apart so that they share no variables, can be resolved if they contain complementary literals. Propositional literals are complementary if one is the negation of the other; first-order literals are complementary if one *unifies with* the negation of the other. Thus, we have

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n}{\text{SUBST}(\theta, \ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)}$$

where $\text{UNIFY}(\ell_i, \neg m_j) = \theta$. For example, we can resolve the two clauses

$$[Animal(F(x)) \vee Loves(G(x), x)] \quad \text{and} \quad [\neg Loves(u, v) \vee \neg Kills(u, v)]$$

by eliminating the complementary literals $Loves(G(x), x)$ and $\neg Loves(u, v)$, with the unifier $\theta = \{u/G(x), v/x\}$, to produce the **resolvent** clause

$$[Animal(F(x)) \vee \neg Kills(G(x), x)].$$

Binary resolution

This rule is called the **binary resolution** rule because it resolves exactly two literals. The

binary resolution rule by itself does not yield a complete inference procedure. The full resolution rule resolves subsets of literals in each clause that are unifiable. An alternative approach is to extend **factoring**—the removal of redundant literals—to the first-order case. Propositional factoring reduces two literals to one if they are *identical*; first-order factoring reduces two literals to one if they are *unifiable*. The unifier must be applied to the entire clause. The combination of binary resolution and factoring is complete.

9.5.3 Example proofs

Resolution proves that $KB \models \alpha$ by proving that $KB \wedge \neg\alpha$ unsatisfiable—that is, by deriving the empty clause. The algorithmic approach is identical to the propositional case, described in Figure 7.13, so we need not repeat it here. Instead, we give two example proofs. The first is the crime example from Section 9.3. The sentences in CNF are

$$\begin{aligned} &\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x) \\ &\neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono) \\ &\neg Enemy(x, America) \vee Hostile(x) \\ &\neg Missile(x) \vee Weapon(x) \\ &Owns(Nono, M_1) \qquad \qquad \qquad Missile(M_1) \\ &American(West) \qquad \qquad \qquad Enemy(Nono, America). \end{aligned}$$

We also include the negated goal $\neg Criminal(West)$. The resolution proof is shown in Figure 9.10. Notice the structure: single “spine” beginning with the goal clause, resolving against clauses from the knowledge base until the empty clause is generated. This is characteristic of resolution on Horn clause knowledge bases. In fact, the clauses along the main spine correspond *exactly* to the consecutive values of the *goals* variable in the backward-chaining algorithm of Figure 9.6. This is because we always choose to resolve with a clause whose positive literal unifies with the leftmost literal of the “current” clause on the spine; this is

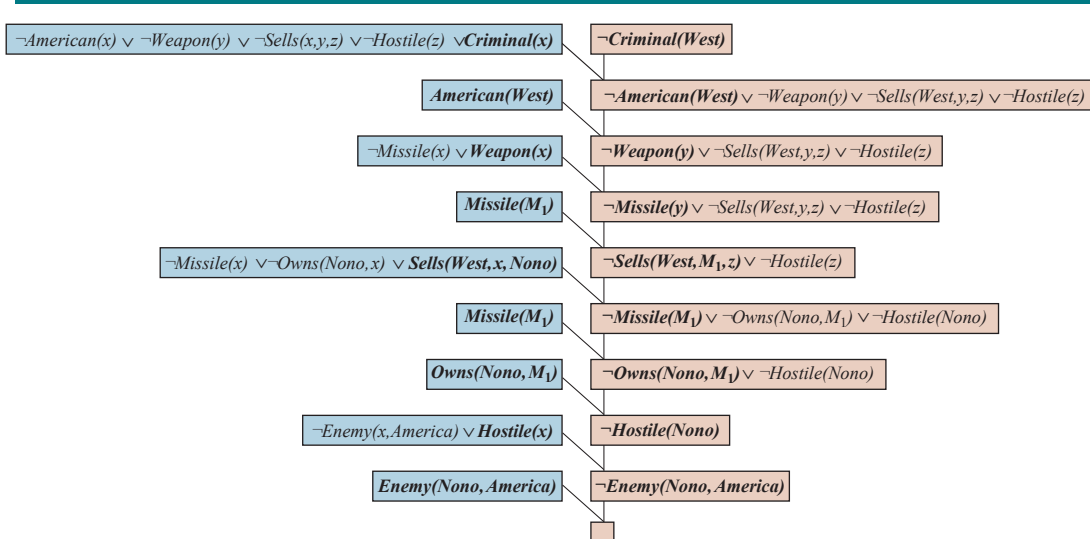


Figure 9.10 A resolution proof that West is a criminal. At each resolution step, the literals that unify are in bold and the clause with the positive literal is shaded blue.

exactly what happens in backward chaining. Thus, backward chaining is just a special case of resolution with a particular control strategy to decide which resolution to perform next.

Our second example makes use of Skolemization and involves clauses that are not definite clauses. This results in a somewhat more complex proof structure. In English:

Everyone who loves all animals is loved by someone.
 Anyone who kills an animal is loved by no one.
 Jack loves all animals.
 Either Jack or Curiosity killed the cat, who is named Tuna.
 Did Curiosity kill the cat?

First, we express the original sentences, some background knowledge, and the negated goal G in first-order logic:

- A. $\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)]$
- B. $\forall x [\exists z \text{ Animal}(z) \wedge \text{Kills}(x, z)] \Rightarrow [\forall y \neg \text{Loves}(y, x)]$
- C. $\forall x \text{ Animal}(x) \Rightarrow \text{Loves}(\text{Jack}, x)$
- D. $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$
- E. $\text{Cat}(\text{Tuna})$
- F. $\forall x \text{ Cat}(x) \Rightarrow \text{Animal}(x)$
- ¬G. $\neg \text{Kills}(\text{Curiosity}, \text{Tuna})$

Now we apply the conversion procedure to convert each sentence to CNF:

- A1. $\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$
- A2. $\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)$
- B. $\neg \text{Loves}(y, x) \vee \neg \text{Animal}(z) \vee \neg \text{Kills}(x, z)$
- C. $\neg \text{Animal}(x) \vee \text{Loves}(\text{Jack}, x)$
- D. $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$
- E. $\text{Cat}(\text{Tuna})$
- F. $\neg \text{Cat}(x) \vee \text{Animal}(x)$
- ¬G. $\neg \text{Kills}(\text{Curiosity}, \text{Tuna})$

The resolution proof that Curiosity killed the cat is given in Figure 9.11. In English, the proof could be paraphrased as follows:

Suppose Curiosity did not kill Tuna. We know that either Jack or Curiosity did; thus Jack must have. Now, Tuna is a cat and cats are animals, so Tuna is an animal. Because anyone who kills an animal is loved by no one, we know that no one loves Jack. On the other hand, Jack loves all animals, so someone loves him; so we have a contradiction. Therefore, Curiosity killed the cat.

The proof answers the question “Did Curiosity kill the cat?” but often we want to pose more general questions, such as “Who killed the cat?” Resolution can do this, but it takes a little more work to obtain the answer. The goal is $\exists w \text{ Kills}(w, \text{Tuna})$, which, when negated, becomes $\neg \text{Kills}(w, \text{Tuna})$ in CNF. Repeating the proof in Figure 9.11 with the new negated goal, we obtain a similar proof tree, but with the substitution $\{w/\text{Curiosity}\}$ in one of the steps. So, in this case, finding out who killed the cat is just a matter of keeping track of the bindings for the query variables in the proof. Unfortunately, resolution can sometimes produce **nonconstructive proofs** for existential goals, where we know a query is true, but there isn’t a unique binding for the variable.

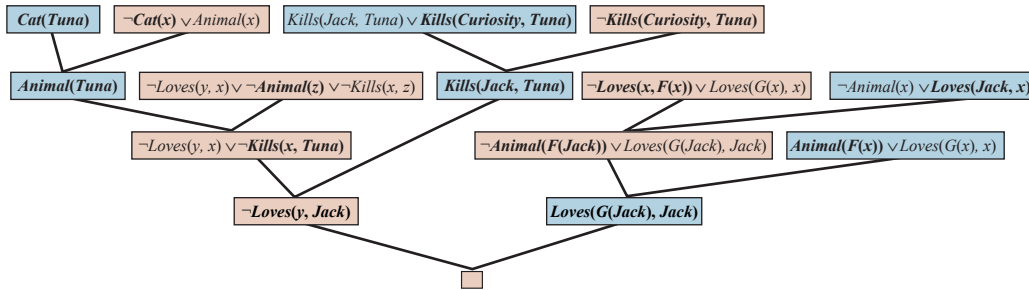


Figure 9.11 A resolution proof that Curiosity killed the cat. Notice the use of factoring in the derivation of the clause $Loves(G(Jack), Jack)$. Notice also in the upper right, the unification of $Loves(x, F(x))$ and $Loves(Jack, x)$ can only succeed after the variables have been standardized apart.

9.5.4 Completeness of resolution

This section gives a completeness proof of resolution. It can be safely skipped by those who are willing to take it on faith.

We show that resolution is **refutation-complete**, which means that *if* a set of sentences is unsatisfiable, then resolution will always be able to derive a contradiction. Resolution cannot be used to generate all logical consequences of a set of sentences, but it can be used to establish that a given sentence is entailed by the set of sentences. Hence, it can be used to find all answers to a given question, $Q(x)$, by proving that $KB \wedge \neg Q(x)$ is unsatisfiable.

Refutation completeness

We take it as given that any sentence in first-order logic (without equality) can be rewritten as a set of clauses in CNF. This can be proved by induction on the form of the sentence, using atomic sentences as the base case (Davis and Putnam, 1960). Our goal therefore is to prove the following: *if S is an unsatisfiable set of clauses, then the application of a finite number of resolution steps to S will yield a contradiction.*



Our proof sketch follows Robinson's original proof with some simplifications from Genesereth and Nilsson (1987). The basic structure of the proof (Figure 9.12) is as follows:

1. First, we observe that if S is unsatisfiable, then there exists a particular set of *ground instances* of the clauses of S such that this set is also unsatisfiable (Herbrand's theorem).
2. We then appeal to the **ground resolution theorem** given in Chapter 7, which states that propositional resolution is complete for ground sentences.
3. We then use a **lifting lemma** to show that, for any propositional resolution proof using the set of ground sentences, there is a corresponding first-order resolution proof using the first-order sentences from which the ground sentences were obtained.

To carry out the first step, we need three new concepts:

- **Herbrand universe:** If S is a set of clauses, then H_S , the Herbrand universe of S , is the set of all ground terms constructible from the following:
 - a. The function symbols in S , if any.
 - b. The constant symbols in S , if any; if none, then a default constant symbol, S .

Herbrand universe

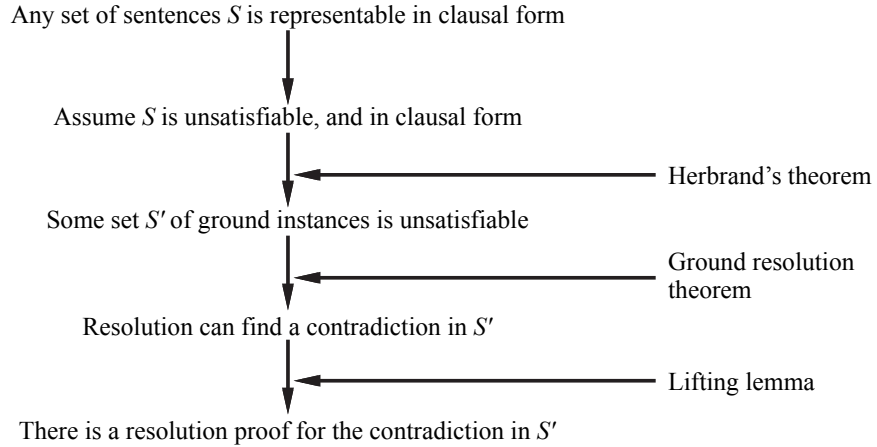


Figure 9.12 Structure of a completeness proof for resolution.

For example, if S contains just the clause $\neg P(x, F(x, A)) \vee \neg Q(x, A) \vee R(x, B)$, then H_S is the following infinite set of ground terms:

$$\{A, B, F(A, A), F(A, B), F(B, A), F(B, B), F(A, F(A, A)), \dots\}.$$

Saturation

- **Saturation:** If S is a set of clauses and P is a set of ground terms, then $P(S)$, the saturation of S with respect to P , is the set of all ground clauses obtained by applying all possible consistent substitutions of ground terms in P for variables in S .

Herbrand base

- **Herbrand base:** The saturation of a set S of clauses with respect to its Herbrand universe is called the Herbrand base of S , written as $H_S(S)$. For example, if S contains solely the clause given above, then $H_S(S)$ is the infinite set of clauses

$$\begin{aligned} &\{\neg P(A, F(A, A)) \vee \neg Q(A, A) \vee R(A, B), \\ &\neg P(B, F(B, A)) \vee \neg Q(B, A) \vee R(B, B), \\ &\neg P(F(A, A), F(F(A, A), A)) \vee \neg Q(F(A, A), A) \vee R(F(A, A), B), \\ &\neg P(F(A, B), F(F(A, B), A)) \vee \neg Q(F(A, B), A) \vee R(F(A, B), B), \dots\} \end{aligned}$$

Herbrand's theorem

These definitions allow us to state a form of **Herbrand's theorem** (Herbrand, 1930):

If a set S of clauses is unsatisfiable, then there exists a finite subset of $H_S(S)$ that is also unsatisfiable.

Let S' be this finite subset of ground sentences. Now, we can appeal to the ground resolution theorem (page 246) to show that the **resolution closure** $RC(S')$ contains the empty clause. That is, running propositional resolution to completion on S' will derive a contradiction.

Now that we have established that there is always a resolution proof involving some finite subset of the Herbrand base of S , the next step is to show that there is a resolution proof using the clauses of S itself, which are not necessarily ground clauses. We start by considering a single application of the resolution rule. Robinson stated this lemma:

Let C_1 and C_2 be two clauses with no shared variables, and let C'_1 and C'_2 be ground instances of C_1 and C_2 . If C' is a resolvent of C'_1 and C'_2 , then there exists a clause C such that (1) C is a resolvent of C_1 and C_2 and (2) C' is a ground instance of C .

Gödel's Incompleteness Theorem

By slightly extending the language of first-order logic to allow for the **mathematical induction schema** in arithmetic, Kurt Gödel was able to show, in his **incompleteness theorem**, that there are true arithmetic sentences that cannot be proved.

The proof of the incompleteness theorem is somewhat beyond the scope of this book, occupying, as it does, at least 30 pages, but we can give a hint here. We begin with the logical theory of numbers. In this theory, there is a single constant, 0, and a single function, S (the successor function). In the intended model, $S(0)$ denotes 1, $S(S(0))$ denotes 2, and so on; the language therefore has names for all the natural numbers. The vocabulary also includes the function symbols $+$, $\times$, and $Expt$ (exponentiation) and the usual set of logical connectives and quantifiers.

The first step is to notice that the set of sentences that we can write in this language can be enumerated. (Imagine defining an alphabetical order on the symbols and then arranging, in alphabetical order, each of the sets of sentences of length 1, 2, and so on.) We can then number each sentence α with a unique natural number $\#_\alpha$ (the **Gödel number**). This is crucial: number theory contains a name for each of its own sentences. Similarly, we can number each possible proof P with a Gödel number $G(P)$, because a proof is simply a finite sequence of sentences.

Now suppose we have a recursively enumerable set A of sentences that are true statements about the natural numbers. Recalling that A can be named by a given set of integers, we can imagine writing in our language a sentence $\alpha(j, A)$ of the following sort:

$\forall i$ i is not the Gödel number of a proof of the sentence whose Gödel number is j , where the proof uses only premises in A .

Then let σ be the sentence $\alpha(\#_\sigma, A)$, that is, a sentence that states its own unprovability from A . (That this sentence always exists is true but not entirely obvious.)

Now we make the following ingenious argument: Suppose that σ is provable from A ; then σ is false (because σ says it cannot be proved). But then we have a false sentence that is provable from A , so A cannot consist of only true sentences—a violation of our premise. Therefore, σ is *not* provable from A . But this is exactly what σ itself claims; hence σ is a true sentence.

So, we have shown (barring $29\frac{1}{2}$ pages) that for any set of true sentences of number theory, and in particular any set of basic axioms, there are other true sentences that *cannot* be proved from those axioms. This establishes, among other things, that we can never prove all the theorems of mathematics *within any given system of axioms*. Clearly, this was an important discovery for mathematics. Its significance for AI has been widely debated, beginning with speculations by Gödel himself. We take up the debate in Chapter 28.

Lifting lemma

This is called a **lifting lemma**, because it lifts a proof step from ground clauses up to general first-order clauses. In order to prove his basic lifting lemma, Robinson had to invent unification and derive all of the properties of most general unifiers. Rather than repeat the proof here, we simply illustrate the lemma:

$$\begin{aligned}
 C_1 &= \neg P(x, F(x, A)) \vee \neg Q(x, A) \vee R(x, B) \\
 C_2 &= \neg N(G(y), z) \vee P(H(y), z) \\
 C'_1 &= \neg P(H(B), F(H(B), A)) \vee \neg Q(H(B), A) \vee R(H(B), B) \\
 C'_2 &= \neg N(G(B), F(H(B), A)) \vee P(H(B), F(H(B), A)) \\
 C' &= \neg N(G(B), F(H(B), A)) \vee \neg Q(H(B), A) \vee R(H(B), B) \\
 C &= \neg N(G(y), F(H(y), A)) \vee \neg Q(H(y), A) \vee R(H(y), B).
 \end{aligned}$$

We see that indeed C' is a ground instance of C . In general, for C'_1 and C'_2 to have any resolvents, they must be constructed by first applying to C_1 and C_2 the most general unifier of a pair of complementary literals in C_1 and C_2 . From the lifting lemma, it is easy to derive a similar statement about any sequence of applications of the resolution rule:

For any clause C' in the resolution closure of S' there is a clause C in the resolution closure of S such that C' is a ground instance of C and the derivation of C is the same length as the derivation of C' .

From this fact, it follows that if the empty clause appears in the resolution closure of S' , it must also appear in the resolution closure of S . This is because the empty clause cannot be a ground instance of any other clause. To recap: we have shown that if S is unsatisfiable, then there is a finite derivation of the empty clause using the resolution rule.

The lifting of theorem proving from ground clauses to first-order clauses provides a vast increase in power. This increase comes from the fact that the first-order proof need instantiate variables only as far as necessary for the proof, whereas the ground-clause methods were required to examine a huge number of arbitrary instantiations.

9.5.5 Equality

None of the inference methods described so far in this chapter can handle an assertion of the form $x = y$ without some additional work. Three distinct approaches can be taken. The first is to axiomatize equality—to write down sentences about the equality relation in the knowledge base. We need to say that equality is reflexive, symmetric, and transitive, and we also have to say that we can substitute equals for equals in any predicate or function. So we need three basic axioms, and then one for each predicate and function:

$$\begin{aligned}
 \forall x \quad x &= x \\
 \forall x, y \quad x &= y \Rightarrow y = x \\
 \forall x, y, z \quad x &= y \wedge y = z \Rightarrow x = z \\
 \forall x, y \quad x &= y \Rightarrow (P_1(x) \Leftrightarrow P_1(y)) \\
 \forall x, y \quad x &= y \Rightarrow (P_2(x) \Leftrightarrow P_2(y)) \\
 &\vdots \\
 \forall w, x, y, z \quad w &= y \wedge x = z \Rightarrow (F_1(w, x) = F_1(y, z)) \\
 \forall w, x, y, z \quad w &= y \wedge x = z \Rightarrow (F_2(w, x) = F_2(y, z)) \\
 &\vdots
 \end{aligned}$$

Given these sentences, a standard inference procedure such as resolution can perform tasks requiring equality reasoning, such as solving mathematical equations. However, these axioms will generate a lot of conclusions, most of them not helpful to a proof. So the second approach is to add inference rules rather than axioms. The simplest rule, **demodulation**, takes a unit clause $x=y$ and some clause α that contains the term x , and yields a new clause formed by substituting y for x within α . It works if the term within α unifies with x ; it need not be exactly equal to x . Note that demodulation is directional; given $x=y$, the x always gets replaced with y , never vice versa. That means that demodulation can be used for simplifying expressions using demodulators such as $z+0=z$ or $z^1=z$. As another example, given

$$\begin{aligned} \text{Father}(\text{Father}(x)) &= \text{PaternalGrandfather}(x) \\ \text{Birthdate}(\text{Father}(\text{Father}(\text{Bella})), 1926) \end{aligned}$$

we can conclude by demodulation

$$\text{Birthdate}(\text{PaternalGrandfather}(\text{Bella}), 1926).$$

More formally, we have

- **Demodulation:** For any terms x , y , and z , where z appears somewhere in literal m_i and where $\text{UNIFY}(x, z) = \theta \neq \text{failure}$, Demodulation

$$\frac{x=y, \quad m_1 \vee \cdots \vee m_n}{\text{SUB}(\text{SUBST}(\theta, x), \text{SUBST}(\theta, y), m_1 \vee \cdots \vee m_n)}.$$

where SUBST is the usual substitution of a binding list, and SUB(x, y, m) means to replace x with y somewhere within m .

The rule can also be extended to handle non-unit clauses in which an equal sign appears:

- **Paramodulation:** For any terms x , y , and z , where z appears somewhere in literal m_i , and where $\text{UNIFY}(x, z) = \theta$, Paramodulation

$$\frac{\ell_1 \vee \cdots \vee \ell_k \vee x=y, \quad m_1 \vee \cdots \vee m_n}{\text{SUB}(\text{SUBST}(\theta, x), \text{SUBST}(\theta, y), \text{SUBST}(\theta, \ell_1 \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_n))}.$$

For example, from

$$P(F(x, B), x) \vee Q(x) \quad \text{and} \quad F(A, y) = y \vee R(y)$$

we have $\theta = \text{UNIFY}(F(A, y), F(x, B)) = \{x/A, y/B\}$, and we can conclude by paramodulation the sentence

$$P(B, A) \vee Q(A) \vee R(B).$$

Paramodulation yields a complete inference procedure for first-order logic with equality.

A third approach handles equality reasoning entirely within an extended unification algorithm. That is, terms are unifiable if they are *provably* equal under some substitution, where “provably” allows for equality reasoning. For example, the terms $1+2$ and $2+1$ normally are not unifiable, but a unification algorithm that knows that $x+y=y+x$ could unify them with the empty substitution. **Equational unification** of this kind can be done with efficient algorithms designed for the particular axioms used (commutativity, associativity, and so on) rather than through explicit inference with those axioms. Theorem provers using this technique are closely related to the CLP systems described in Section 9.4. Equational unification

9.5.6 Resolution strategies

We know that repeated applications of the resolution inference rule will eventually find a proof if one exists. In this subsection, we examine strategies that help find proofs *efficiently*.

Unit preference

Unit preference: This strategy prefers to do resolutions where one of the sentences is a single literal (also known as a **unit clause**). The idea behind the strategy is that we are trying to produce an empty clause, so it might be a good idea to prefer inferences that produce shorter clauses. Resolving a unit sentence (such as P) with any other sentence (such as $\neg P \vee \neg Q \vee R$) always yields a clause (in this case, $\neg Q \vee R$) that is shorter than the other clause. When the unit preference strategy was first tried for propositional inference in 1964, it led to a dramatic speedup, making it feasible to prove theorems that could not be handled without the preference. **Unit resolution** is a restricted form of resolution in which every resolution step must involve a unit clause. Unit resolution is incomplete in general, but complete for Horn clauses. Unit resolution proofs on Horn clauses resemble forward chaining.

The OTTER theorem prover (McCune, 1990), uses a form of best-first search. Its heuristic function measures the “weight” of each clause, where lighter clauses are preferred. The exact choice of heuristic is up to the user, but generally, the weight of a clause should be correlated with its size or difficulty. Unit clauses are treated as light; the search can thus be seen as a generalization of the unit preference strategy.

Set of support

Set of support: Preferences that try certain resolutions first are helpful, but in general it is more effective to try to eliminate some potential resolutions altogether. For example, we can insist that every resolution step involve at least one element of a special set of clauses—the *set of support*. The resolvent is then added into the set of support. If the set of support is small relative to the whole knowledge base, the search space will be reduced dramatically.

To ensure completeness of this strategy, we can choose the set of support S so that the remainder of the sentences are jointly satisfiable. For example, one can use the negated query as the set of support, on the assumption that the original knowledge base is consistent. (After all, if it is not consistent, then the fact that the query follows from it is vacuous.) The set-of-support strategy has the additional advantage of generating goal-directed proof trees that are often easy for humans to understand.

Input resolution

Input resolution: In this strategy, every resolution combines one of the input sentences (from the KB or the query) with some other sentence. The proof in Figure 9.10 on page 319 uses only input resolutions and has the characteristic shape of a single “spine” with single sentences combining onto the spine. Clearly, the space of proof trees of this shape is smaller than the space of all proof graphs. In Horn knowledge bases, Modus Ponens is a kind of input resolution strategy, because it combines an implication from the original KB with some other sentences. Thus, it is no surprise that input resolution is complete for knowledge bases that are in Horn form, but incomplete in the general case. The **linear resolution** strategy is a slight generalization that allows P and Q to be resolved together either if P is in the original KB or if P is an ancestor of Q in the proof tree. Linear resolution is complete.

Linear resolution

Subsumption

Subsumption: The subsumption method eliminates all sentences that are subsumed by (that is, more specific than) an existing sentence in the KB. For example, if $P(x)$ is in the KB, then there is no sense in adding $P(A)$ and even less sense in adding $P(A) \vee Q(B)$. Subsumption helps keep the KB small and thus helps keep the search space small.

Learning: We can improve a theorem prover by learning from experience. Given a collection of previously-proved theorems, train a machine learning system to answer the question: given a set of premises and a goal to prove, what proof steps are similar to steps that were successful in the past? The DEEPHOL system (Bansal *et al.*, 2019) does exactly that, using deep neural networks (see Chapter 22) to build models (called *embeddings*) of goals and premises, and using them to make selections. Training can use both human- and computer-generated proofs as examples, starting from a collection of 10,000 proofs.

Learning

Practical uses of resolution theorem provers

We have shown how first-order logic can represent a simple real-world scenario involving concepts like selling, weapons, and citizenship. But complex real-world scenarios have too much uncertainty and too many unknowns. Logic has proven to be more successful for scenarios involving formal, strictly defined concepts, such as the **synthesis** and **verification** of both hardware and software. Theorem-proving research is carried out in the fields of hardware design, programming languages, and software engineering—not just in AI.

Synthesis

Verification

In the case of hardware, the axioms describe the interactions between signals and circuit elements. (See Section 8.4.2 on page 291 for an example.) Logical reasoners designed specially for verification have been able to verify entire CPUs, including their timing properties (Srivasa and Bickford, 1990). The AURA theorem prover has been applied to design circuits that are more compact than any previous design (Wojciechowski and Wojcik, 1983).

In the case of software, reasoning about programs is quite similar to reasoning about actions, as in Chapter 7: axioms describe the preconditions and effects of each statement. The formal synthesis of algorithms was one of the first uses of theorem provers, as outlined by Cordell Green (1969a), who built on earlier ideas by Herbert Simon (1963). The idea is to constructively prove a theorem to the effect that “there exists a program p satisfying a certain specification.” Although fully automated **deductive synthesis**, as it is called, has not yet become feasible for general-purpose programming, hand-guided deductive synthesis has been successful in designing several novel and sophisticated algorithms. Synthesis of special-purpose programs, such as scientific computing code, is also an active area of research.

Similar techniques are now being applied to software verification by systems such as the SPIN model checker (Holzmann, 1997). For example, the Remote Agent spacecraft control program was verified before and after flight (Havelund *et al.*, 2000). The RSA public key encryption algorithm and the Boyer–Moore string-matching algorithm have been verified this way (Boyer and Moore, 1984).

Summary

We have presented an analysis of logical inference in first-order logic and a number of algorithms for doing it.

- A first approach uses inference rules (**universal instantiation** and **existential instantiation**) to **propositionalize** the inference problem. Typically, this approach is slow, unless the domain is small.
- The use of **unification** to identify appropriate substitutions for variables eliminates the instantiation step in first-order proofs, making the process more efficient in many cases.

- A lifted version of **Modus Ponens** uses unification to provide a natural and powerful inference rule, **generalized Modus Ponens**. The **forward-chaining** and **backward-chaining** algorithms apply this rule to sets of definite clauses.
- Generalized Modus Ponens is complete for definite clauses, although the entailment problem is **semidecidable**. For **Datalog** knowledge bases consisting of function-free definite clauses, entailment is decidable.
- Forward chaining is used in **deductive databases**, where it can be combined with relational database operations. It is also used in **production systems**, which perform efficient updates with very large rule sets. Forward chaining is complete for Datalog and runs in polynomial time.
- Backward chaining is used in **logic programming systems**, which employ sophisticated compiler technology to provide very fast inference. Backward chaining suffers from redundant inferences and infinite loops; these can be alleviated by **memoization**.
- **Prolog**, unlike first-order logic, uses a closed world with the unique names assumption and negation as failure. These make Prolog a more practical programming language, but bring it further from pure logic.
- The generalized **resolution** inference rule provides a complete proof system for first-order logic, using knowledge bases in conjunctive normal form.
- Several strategies exist for reducing the search space of a resolution system without compromising completeness. One of the most important issues is dealing with equality; we showed how **demodulation** and **paramodulation** can be used.
- Efficient resolution-based theorem provers have been used to prove interesting mathematical theorems and to verify and synthesize software and hardware.

Bibliographical and Historical Notes

Gottlob Frege, who developed full first-order logic in 1879, based his system of inference on a collection of valid schemas plus a single inference rule, Modus Ponens. Whitehead and Russell (1910) expounded the so-called *rules of passage* (the actual term is from Herbrand (1930)) that are used to move quantifiers to the front of formulas. Skolem constants and Skolem functions were introduced, appropriately enough, by Thoralf Skolem (1920). Oddly enough, it was Skolem who introduced the Herbrand universe (Skolem, 1928).

Herbrand's theorem (Herbrand, 1930) has played a vital role in the development of automated reasoning. Herbrand is also the inventor of **unification**. Gödel (1930) built on the ideas of Skolem and Herbrand to show that first-order logic has a complete proof procedure. Alan Turing (1936) and Alonzo Church (1936) simultaneously showed, using very different proofs, that validity in first-order logic was not decidable. The excellent text by Enderton (1972) explains all of these results in a rigorous yet understandable fashion.

Abraham Robinson proposed that an automated reasoner could be built using propositionalization and Herbrand's theorem, and Paul Gilmore (1960) wrote the first program. Davis and Putnam (1960) introduced the propositionalization method of Section 9.1. Prawitz (1960) developed the key idea of letting the quest for propositional inconsistency drive the search, and generating terms from the Herbrand universe only when they were necessary to estab-

lish propositional inconsistency. This idea led John Alan Robinson (no relation) to develop resolution (Robinson, 1965).

Resolution was adopted for question-answering systems by Cordell Green and Bertram Raphael (1968). Early AI implementations put a good deal of effort into data structures that would allow efficient retrieval of facts; this work is covered in AI programming texts (Charniak *et al.*, 1987; Norvig, 1992; Forbus and de Kleer, 1993). By the early 1970s, **forward chaining** was well established in AI as an easily understandable alternative to resolution. AI applications typically involved large numbers of rules, so it was important to develop efficient rule-matching technology, particularly for incremental updates.

The technology for **production systems** was developed to support such applications. The production system language OPS-5 (Forgy, 1981; Brownston *et al.*, 1985), incorporating the efficient Rete match process (Forgy, 1982), was used for applications such as the R1 expert system for minicomputer configuration (McDermott, 1982). Kraska *et al.* (2017) describe how neural nets can learn an efficient indexing scheme for specific data sets.

The SOAR cognitive architecture (Laird *et al.*, 1987; Laird, 2008) was designed to handle very large rule sets—up to a million rules (Doorenbos, 1994). Example applications of SOAR include controlling simulated fighter aircraft (Jones *et al.*, 1998), airspace management (Taylor *et al.*, 2007), AI characters for computer games (Wintermute *et al.*, 2007), and training tools for soldiers (Wray and Jones, 2005).

The field of **deductive databases** began with a workshop in Toulouse in 1977 attended by experts in logical inference and databases (Gallaire and Minker, 1978). Influential work by Chandra and Harel (1980) and Ullman (1985) led to the adoption of **Datalog** as a standard language for deductive databases. The development of the **magic sets** technique for rule rewriting by Bancilhon *et al.* (1986) allowed forward chaining to borrow the advantage of goal-directedness from backward chaining.

The rise of the Internet led to increased availability of massive online databases. This drove increased interest in integrating multiple databases into a consistent dataspace (Halevy, 2007). Kraska *et al.* (2017) showed speedups of up to 70% by using machine learning to create **learned index structures** for efficient data lookup.

Backward chaining for logical inference originated in the PLANNER language (Hewitt, 1969). Meanwhile, in 1972, Alain Colmerauer had developed and implemented **Prolog** for the purpose of parsing natural language—Prolog's clauses were intended initially as context-free grammar rules (Roussel, 1975; Colmerauer *et al.*, 1973).

Much of the theoretical background for logic programming was developed by Robert Kowalski at Imperial College London, working with Colmerauer; see Kowalski (1988) and Colmerauer and Roussel (1993) for a historical overview. Efficient Prolog compilers are generally based on the Warren Abstract Machine (WAM) model of computation developed by David H. D. Warren (1983). Van Roy (1990) showed that Prolog programs can be competitive with C programs in terms of speed.

Methods for avoiding unnecessary looping in recursive logic programs were developed independently by Smith *et al.* (1986) and Tamaki and Sato (1986). The latter paper also included memoization for logic programs, a method developed extensively as **tabled logic programming** by David S. Warren. Swift and Warren (1994) show how to extend the WAM to handle tabling, enabling Datalog programs to execute an order of magnitude faster than forward-chaining deductive database systems.

Early work on constraint logic programming was done by Jaffar and Lassez (1987). Jaffar *et al.* (1992) developed the CLP(R) system for handling real-valued constraints. There are now commercial products for solving large-scale configuration and optimization problems with constraint programming; one of the best known is ILOG (Junker, 2003). Answer set programming (Gelfond, 2008) extends Prolog, allowing disjunction and negation.

Texts on logic programming and Prolog include Shoham (1994), Bratko (2009), Clocksin (2003), and Clocksin and Mellish (2003). Prior to 2000, the *Journal of Logic Programming* was the journal of record; it has been replaced by *Theory and Practice of Logic Programming*. Logic programming conferences include the International Conference on Logic Programming (ICLP) and the International Logic Programming Symposium (ILPS).

Research into **mathematical theorem proving** began even before the first complete first-order systems were developed. Herbert Gelernter's Geometry Theorem Prover (Gelernter, 1959) used heuristic search methods combined with diagrams for pruning false subgoals and was able to prove some quite intricate results in Euclidean geometry. The **demodulation** and **paramodulation** rules for equality reasoning were introduced by Wos *et al.* (1967) and Wos and Robinson (1968), respectively. These rules were also developed independently in the context of term-rewriting systems (Knuth and Bendix, 1970). The incorporation of equality reasoning into the unification algorithm is due to Gordon Plotkin (1972). Jouannaud and Kirchner (1991) survey equational unification from a term-rewriting perspective. An overview of unification is given by Baader and Snyder (2001).

A number of control strategies have been proposed for resolution, beginning with the unit preference strategy (Wos *et al.*, 1964). The set-of-support strategy was proposed by Wos *et al.* (1965) to provide a degree of goal-directedness in resolution. Linear resolution first appeared in Loveland (1970). Genesereth and Nilsson (1987, Chapter 5) provide an analysis of a wide variety of control strategies. Alemi *et al.* (2017) show how the DEEPMATH system uses deep neural nets to select the axioms that are most likely to lead to a proof when handed to a traditional theorem prover. In a sense, the neural net plays the role of the mathematician's intuition, and the theorem prover plays the role of the mathematician's technical expertise. (Loos *et al.*, 2017) show that this approach can be extended to help guide the search, allowing more theorems to be proved.

A Computational Logic (Boyer and Moore, 1979) is the basic reference on the Boyer-Moore theorem prover. Stickel (1992) describes the Prolog Technology Theorem Prover (PTTP), which combines Prolog compilation and model elimination. SETHEO (Letz *et al.*, 1992) is another widely used theorem prover based on this approach. LEANTAP (Beckert and Posegga, 1995) is an efficient theorem prover implemented in only 25 lines of Prolog. Weidenbach (2001) describes SPASS, one of the strongest current theorem provers. The most successful theorem prover in recent annual competitions has been VAMPIRE (Riazanov and Voronkov, 2002). The COQ system (Bertot *et al.*, 2004) and the E equational solver (Schulz, 2004) have also proven to be valuable tools for proving correctness.

Theorem provers have been used to automatically synthesize and verify software. Examples include the control software for NASA's Orion capsule (Lowry, 2008) and other spacecraft (Denney *et al.*, 2006). The design of the FM9001 32-bit microprocessor was proved correct by the NQTHM theorem proving system (Hunt and Brock, 1992).

The Conference on Automated Deduction (CADE) runs an annual contest for automated theorem provers. Sutcliffe (2016) describes the 2016 competition; top-scoring systems in-

clude VAMPIRE (Riazanov and Voronkov, 2002), PROVER9 (Sabri, 2015), and an updated version of E (Schulz, 2013). Wiedijk (2003) compares the strength of 15 mathematical provers. TPTP (Thousands of Problems for Theorem Provers) is a library of theorem-proving problems, useful for comparing the performance of systems (Sutcliffe and Suttner, 1998; Sutcliffe *et al.*, 2006).

Theorem provers have come up with novel mathematical results that eluded human mathematicians for decades, as detailed in the book *Automated Reasoning and the Discovery of Missing Elegant Proofs* (Wos and Pieper, 2003). The SAM (Semi-Automated Mathematics) program was the first, proving a lemma in lattice theory (Guard *et al.*, 1969). The AURA program has also answered open questions in several areas of mathematics (Wos and Winker, 1983). The Boyer–Moore theorem prover (Boyer and Moore, 1979) was used by Natarajan Shankar to construct a formal proof of Gödel’s Incompleteness Theorem (Shankar, 1986). The NUPRL system proved Girard’s paradox (Howe, 1987) and Higman’s Lemma (Murthy and Russell, 1990).

In 1933, Herbert Robbins proposed a simple set of axioms—the **Robbins algebra**—[Robbins algebra](#) that appeared to define Boolean algebra, but no proof could be found (despite serious work by Alfred Tarski and others) until EQP (a version of OTTER) computed a proof (McCune, 1997). Benzmüller and Paleo (2013) used a higher-order theorem prover to verify Gödel’s proof of the existence of “God.” The Kepler sphere-packing theorem was proved by Thomas Hales (2005) with the help of some complicated computer calculations, but the proof was not completely accepted until a formal proof was generated with the help of the HOL Light and Isabelle proof assistants (Hales *et al.*, 2017).

Many early papers in mathematical logic are collected in *From Frege to Gödel: A Source Book in Mathematical Logic* (van Heijenoort, 1967). Textbooks geared toward automated deduction include the classic *Symbolic Logic and Mechanical Theorem Proving* (Chang and Lee, 1973), as well as more recent works by Duffy (1991), Wos *et al.* (1992), Bibel (1993), and Kaufmann *et al.* (2000). The principal journal for theorem proving is the *Journal of Automated Reasoning*; the main conferences are the annual Conference on Automated Deduction (CADE) and the International Joint Conference on Automated Reasoning (IJCAR). The *Handbook of Automated Reasoning* (Robinson and Voronkov, 2001) collects papers in the field. MacKenzie’s *Mechanizing Proof* (2004) covers the history and technology of theorem proving for the popular audience.